Politecnico di Torino

ensil-ensci ÉCOLE D'INGÉNIEURS DE LIMOGES

Internship report
4th year of Engineering School
Academic year 2025 / 2026

Risk-Aware MPPI for UGV Navigation

Vision-Based Terrain Risk Estimation and Sampling-Based Control on a TurtleBot 4

Student: Justin Amen N'GBLOGNI

Company supervisors: Prof. Fausto Francesco Lizzio
Prof. Elisa Capello

Host institution: DIMEAS, Politecnico di Torino (Italy)

Université de Limoges

Speciality: Mechatronics

Acknowledgements

I would like to thank Prof. Elisa Capello for giving me the opportunity to carry out this internship at DIMEAS, Politecnico di Torino, for welcoming me into her team, and for entrusting me to Prof. Fausto Francesco Lizzio's supervision.

I am sincerely grateful to my main supervisor, Prof. Fausto Francesco Lizzio, for his guidance, availability, and valuable feedback throughout the internship.

I would also like to thank Riccardo Enrico and Petre Ricioppo, PhD students at DIMEAS, for their day-to-day support, for sharing their experience with the robotic platform and the simulation environment, and for their help whenever I ran into a technical difficulty.

Finally, I thank everyone at DIMEAS who, directly or indirectly, contributed to making this internship a valuable learning experience.

Abstract

Autonomous ground robots that navigate structured indoor environments must avoid hazards that a 2D LiDAR simply cannot perceive: wet, mud, tiles, thick carpet, and other surfaces that are geometrically flat but physically dangerous. This internship addresses that perception gap by building a *risk-aware* navigation stack for a TurtleBot 4 and validating it in simulation and in a hybrid real/virtual experiment.

The system has two parts. First, a vision-based **terrain risk map** fuses a semantic layer: a lightweight segmentation network that turns each camera pixel into an expected risk $R(\boldsymbol{p}) = \sum_c r_c \, \mathbb{P}(c \mid \boldsymbol{p})$, with a geometric layer that recovers walls and obstacles from the depth point cloud. The fused risk is accumulated into a persistent bird's-eye-view (BEV) costmap. Second, a risk-aware **Model Predictive Path Integral (MPPI)** controller, running thousands of GPU-sampled rollouts, treats this costmap as a continuous cost field and trades off progress toward the goal against accumulated terrain risk. A single parameter, λ_{risk}, tunes the robot from risk-blind (standard MPPI baseline) to strongly risk-averse.

The controller was analysed quantitatively: an explicit cost-arbitrage model predicts, for each hazard, the λ_{risk} threshold at which the robot switches from crossing a risky patch to detouring around it, and validated on a set of controlled scenarios (forced crossing, slalom, cross-or-detour). The evaluation relies on a single interface, the risk costmap, which lets a *learned* map (from the camera, in a Gazebo world) and a *ground-truth* map (baked from known scene geometry) be used interchangeably by the same controller. As a final demonstration, the controller was run on the physical TurtleBot 4, which navigated a *virtual* risk field baked into the costmap, steering around hazards that existed only in the map. Closing the loop with on-board camera perception *on the real robot* was attempted but not completed, and is left as future work.

Across two controlled scenarios, tuning λ_{risk} from the risk-blind baseline to a risk-aware setting cut ground-truth risk exposure by **44%** on a forced wall-to-wall crossing and by **100%** on an open-field detourable patch, at a path-length and time cost of only 2–9%. A quantitative cost-arbitrage model, corrected for the controller's actual goal-distance weight, predicts the measured crossing-to-detour threshold to within the right order of magnitude, and the campaign also identifies an upper λ_{risk} regime where added caution buys no further safety and instead destabilises the controller into indefinite hesitation.

Keywords: risk-aware navigation, traversability, MPPI, semantic segmentation, RGB-D perception, ROS 2, TurtleBot 4, sampling-based control.

Contents

List of Figures

List of Tables

CHAPTER 1

Introduction

1.1 Context and Motivation

Mobile robots are increasingly deployed in human-centred indoor spaces, offices, warehouses, laboratories, entrance halls. In these environments the floor is not uniformly safe: a robot may encounter a wet tile after cleaning, a patch of tracked-in mud, a thick rug, a door threshold, or a staircase. A standard navigation stack built around a 2D LiDAR treats all of these as free space, because the LiDAR beam sweeps a single horizontal plane and only reacts to rigid, vertical obstacles. It cannot distinguish a dry tile from a wet one, and it perceives a staircase only as its first edge.

Figure 1.1: A robot in the hallway in front of a cardboard box and a puddle of water

The perception gap this project addresses

Mud, water, wet gravel, damp concrete, thick carpet, and unstable ground are *visually distinct* but all equally "free" to a 2D LiDAR. Bridging this gap, giving the robot a sense of **how risky the ground ahead is**, not just **whether something blocks it**, is the core objective of the internship.

The platform used throughout the project is a **TurtleBot 4** (iRobot Create 3 base, Raspberry Pi 4, OAK-D Pro RGB-D camera, RPLIDAR A1). It is a small wheeled UGV(Unmanned Ground Vehicle) with a ground clearance of only a few centimetres and a top speed of about $0.31\,\mathrm{m\,s^{-1}}$, which makes it especially sensitive to surface hazards. The camera is what closes the perception gap: it sees texture and colour, exactly what the LiDAR cannot.

1.2 Problem Statement

The problem tackled here can be stated compactly:

Given an RGB-D stream from a moving UGV, estimate a dense, spatial traversability risk map

of the ground, and use it inside a sampling-based motion planner so that the robot reaches its goal while actively minimising its exposure to hazardous terrain.

This decomposes into two coupled sub-problems: (i) *perception*, turning images and depth into a metric risk field $R(x, y) \in [0, 1]$ expressed in a world-fixed frame; and (ii) *control*, planning trajectories that are cheap in both distance-to-goal and accumulated risk. The two are connected by a bird's-eye-view (BEV) costmap that the perception module publishes and the controller consumes.

1.3 Objectives

The internship was organised around five phases:

1. **State of the art & problem definition**, survey traversability estimation (geometric, semantic, self-supervised), define the indoor urban hazards relevant to the TurtleBot 4, and fix the traversability criteria.
2. **Simulation environment**: build ROS 2 / Gazebo worlds with controllable hazards, integrate the RGB-D + IMU sensor suite, and set up a dataset-generation pipeline.
3. **Vision-based risk learning**: train a lightweight segmentation network that predicts a dense risk map, and evaluate its quality and inference speed.
4. **Spatial projection & MPPI integration**: project the image-space risk onto the ground plane into $R(x, y)$, and integrate it as a cost term inside an MPPI controller.
5. **Experimental validation**: compare standard MPPI against the risk-aware version, analyse the robot's behaviour near hazards, and run a hybrid real/virtual demonstration on the physical robot.

1.4 Contributions

The concrete outcomes of the internship are:

- A **two-layer terrain risk map** (semantic + geometric) with a real-time ROS 2 implementation that publishes a persistent, world-sized BEV costmap (Chapter 4).
- A comparison of three segmentation backbones (SegFormer-B0, DeepLabV3+/MobileNetV2, YOLO26) for the semantic layer, trading accuracy against embedded-inference speed.
- A **GPU risk-aware MPPI controller** built on the MPPI-Generic library, with a custom CUDA cost function that samples the risk map from a hardware texture, and live diagnostics (rollout visualisation, effective sample size) (Chapter 5).
- A **quantitative cost-arbitrage model** that predicts, for a given hazard and geometry, the λ_{risk} threshold at which the optimal behaviour switches from crossing to detouring, turning controller tuning into a calculation rather than trial and error.
- A suite of **controlled evaluation scenarios** and an automated metrics logger, and a demonstration on the **physical robot**, which navigates a *virtual* risk field baked into the costmap, avoiding hazards that exist only in the map (Chapter 6).

1.5 Report Outline

Chapter 2 reviews traversability estimation and the MPPI control framework. Chapter 3 describes the robot platform and the end-to-end system architecture. Chapter 4 details the perception pipeline and the risk map. Chapter 5 presents the risk-aware MPPI controller and its tuning theory. Chapter 6 describes the simulation worlds, scenarios, evaluation methodology, and the hybrid real/virtual test. Chapter 7 reports and discusses the results. Chapter 9 concludes and outlines future work.

CHAPTER 2

Background and Related Work

This chapter reviews the two bodies of work the project builds on: methods for estimating *terrain traversability*, and the *Model Predictive Path Integral* (MPPI) control framework used to act on that estimate.

2.1 Traversability Estimation

Traversability refers to a robot's ability to cross a region of terrain safely; it captures the difficulty of driving through a region given the terrain's physical properties (slope, roughness, surface condition, and so on) [1]. Estimating it is the central perception problem of this project. The literature distinguishes three main families of methods.

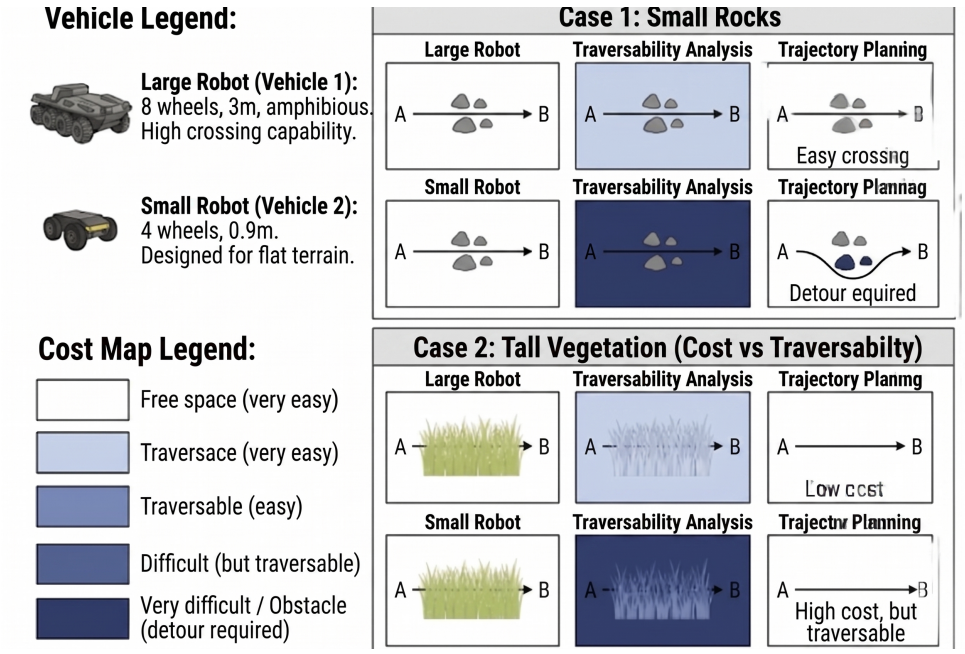


Figure 2.1: Different robots facing different types of obstacles.

2.1.1 Geometric Methods

Geometric methods use depth data (LiDAR or depth camera) to analyse the *shape* of the terrain: if the local slope is too steep, the surface too irregular, or an obstacle is detected above a height threshold, the terrain is declared non-traversable.

Criterion	Detail
Data used	3D point cloud (LiDAR or RGB-D)
Risks detected	Solid obstacles, steep slopes, elevation changes
Advantages	Robust, fast, no training labels required
Limitations	Cannot detect mud, water, gravel, flat but dangerous surfaces

Relevance

The TurtleBot 4's RPLIDAR A1 provides *only* 2D geometric data (a single horizontal plane). This is precisely the gap the project bridges with the camera. The geometric *layer* of our risk map (Chapter 4) recovers walls and obstacles, but from the depth camera rather than the LiDAR.

2.1.2 Semantic Methods

Semantic methods use a camera and a neural network to recognise *surface categories* (tile, carpet, mud, water, ...). Each pixel is classified, and each category is assigned a risk score. This is the **main approach** of the project. Its central formulation, adopted throughout, is:

Semantic risk of a pixel

$$R(\boldsymbol{p}) = \sum_{c} r_c \, \mathbb{P}(c \mid \boldsymbol{p})$$

$\boldsymbol{p}$: position of a pixel (later, of a ground point)
c : terrain category (tile, mud, wet, ...)
$r_c \in [0,1]$: expert-assigned risk score of category c
$\mathbb{P}(c \mid \boldsymbol{p})$: probability that $\boldsymbol{p}$ belongs to c (softmax of the network)

The result is a *dense, continuous* risk image, not a hard binary mask. The main advantage is interpretability: we know *why* a region is avoided (it is mud, not simply "high cost"). The main limitation is that the scores r_c are set **by hand**, and two physically different surfaces sharing a label receive the same score.

2.1.3 Self-Supervised Learning Methods

Self-supervised methods learn to estimate risk *without human labels*, using the **robot's own experience**: a surface that has been crossed without slipping or shaking is likely traversable, while slip events and abnormal IMU signatures serve as negative signals. STERLING [2] is a representative recent example: it pulls together visual and proprioceptive representations of the same terrain with a non-contrastive objective (VICReg), producing a terrain representation usable for classification and risk scoring with no manual labelling.

Relevance

The TurtleBot 4 already exposes everything a self-supervised approach needs, IMU on `/imu`, wheel odometry on `/odom`, and the OAK-D camera, with no hardware change. This project uses the supervised semantic approach for speed of implementation, and identifies self-supervision as the natural extension (Chapter 9).

2.1.4 Summary

Table 2.1 contrasts the three families. The semantic approach is primary for this project; the geometric layer complements it for rigid obstacles; self-supervision is a future extension.

Table 2.1: Comparison of traversability estimation families.

Criterion	Geometric	Semantic	Self-supervised
Solid-obstacle detection	Yes	No	No
Mud/water/carpet detection	No	Yes	Partial
Requires labels	No	Yes	No
TurtleBot 4 sensor	LiDAR	OAK-D	IMU+Cam
Role in this project	Layer 2	**Primary**	Extension

2.2 Model Predictive Path Integral Control

Model Predictive Path Integral Control (MPPI) is a sampling-based, gradient-free model-predictive controller derived from stochastic optimal control and information theory [3]. At each control step it:

1. samples K control sequences by perturbing a warm-started nominal sequence with noise, $u_k = u_{\text{nom}} + \delta u_k$;
2. rolls each one forward through the system dynamics over a horizon of T steps, obtaining K candidate trajectories;
3. evaluates a state-dependent cost S_k for each trajectory;
4. combines the samples with an exponential (softmax) weighting $w_k = \exp\big(-(S_k - S_{\min})/\lambda_{\text{temp}}\big)$ and outputs the weighted-average control $u_{\text{new}} = \sum_k w_k u_k / \sum_k w_k$;
5. applies the first control, then slides the sequence one step for the next cycle (warm start).

Because it never linearises the dynamics or the cost, MPPI accommodates the *arbitrary, non-differentiable* cost fields that a risk map produces, and it is massively parallel: the K rollouts are independent and map naturally onto a GPU. These properties make it a strong fit for risk-aware navigation.

The two roles of the temperature λ

In the information-theoretic derivation, λ appears *twice*: in the softmax weighting above, and in a control cost $\lambda(1-\alpha)\,u^\top \Sigma^{-1} u$ that guarantees the consistency of the importance sampling. Setting $\alpha = 1$ cancels the second term, giving a pure "cost-shaping" MPPI in which trajectory smoothness comes entirely from the (colored) sampling noise. This project uses $\alpha = 1$; the consequence is

discussed in Section 5.5.

Why MPPI for this project

The risk map is a dense, non-convex, frequently-updated cost field with no useful gradient. MPPI turns it directly into behaviour without any convex approximation, and its rollout cloud can be visualised, which, as Chapter 5 shows, made the controller's decisions near hazards legible and debuggable.

CHAPTER 3

Platform and System Overview

3.1 The TurtleBot 4 Platform

Figure 3.1: TurtleBot 4 (Standard)

Table 3.1: TurtleBot 4 (Standard) platform summary.

Component	Description
Mobile base	iRobot Create 3, differential drive, up to 0.306 $\mathrm{m\,s^{-1}}$
Computer	Raspberry Pi 4B, Ubuntu + ROS 2
Camera	OAK-D Pro (Luxonis), RGB-D stereo, embedded VPU
LiDAR	RPLIDAR A1, 2D, 360°, 0.15 m to 12 m
Base sensors	IMU, wheel encoders, optical ground sensor, cliff/bump
ROS interface	all sensors publish over ROS 2 topics

The **OAK-D Pro** is the central sensor: an RGB-D stereo camera whose colour and depth images are pixel-aligned, so a risk value computed on an RGB pixel attaches directly to the 3D point deprojected

from the corresponding depth pixel. The **RPLIDAR A1** confirms the 360° rigid structure of the scene but, being a single horizontal plane, sees neither the ground surface nor low obstacles, the motivation of Chapter 1.

Scoping: indoor urban context

The TurtleBot 4 has a ground clearance of $\approx 3\,\text{cm}$ and a top speed of $0.306\,\text{m}\,\text{s}^{-1}$. The hazards considered here are those of **structured indoor urban environments** (corridors, labs, entrance halls): wet tiles, mud/thick carpet, thresholds, small steps, and staircases. Deep outdoor mud, gravel, and standing water are out of scope for this platform.

3.2 Simulation Environment

Before describing the software architecture, it is worth introducing where the robot actually operates, since the dataset used to train the perception model (Chapter 4) was captured directly in this simulated setting. The simulation uses **Gazebo** (Harmonic) with a full TurtleBot 4 model, its RGB-D camera and LiDAR bridged to ROS 2, and a ground-truth pose used to anchor the world frame (Section 3.4). Several worlds were built:

- `warehouse_risk`: a multi-room world with mud, wet, small-step, and staircase patches, used for the initial integration and dataset capture.
- `cage_risk`: a bare $6 \times 4\,\text{m}$ arena (the dimensions of the real test cage), used for the controlled scenarios so that nothing but the scenario under test is present.
- `sequential_risk`: a corridor with two consecutive risk strips, used during the early real-robot integration tests (Section 6.3).

Risk zones are Gazebo models (wet areas in blue, mud in brown, and tile in gray) plus everyday obstacles (box, backpack, chair) for the realistic scenario.

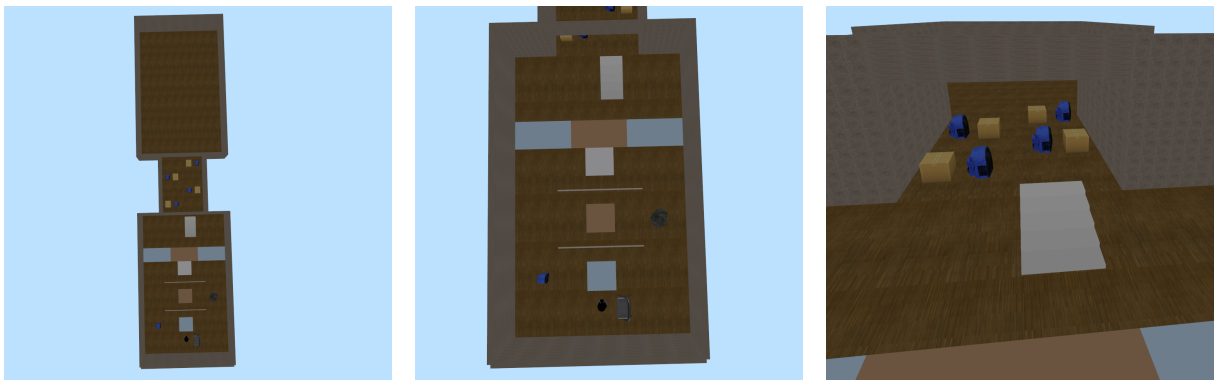


Figure 3.2: The `warehouse_risk` Gazebo world: top-down view of the full corridor and rooms (left), a room with wet/mud/small-step patches (centre), and a first-person view among everyday obstacles (right).

3.3 End-to-End Architecture

Figure 3.3 shows the full pipeline. Two ROS 2 packages implement it: `risk_mppi_sim` (Python: perception, simulation, evaluation) and `risk_mppi_controller` (C++/CUDA: the MPPI node).

The pipeline is:

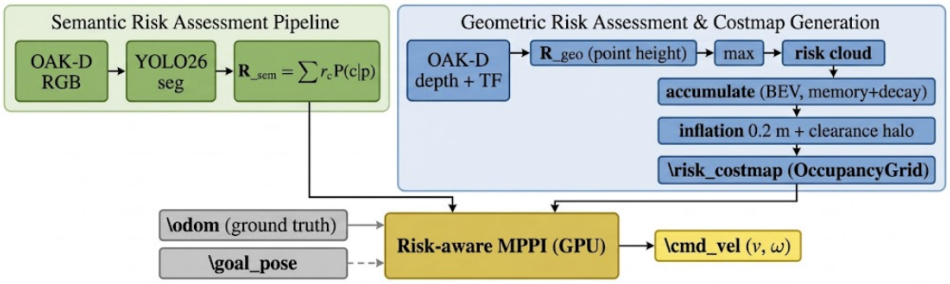


Figure 3.3: Data flow of the risk-aware navigation stack.

1. The **perception node** (`risk_fusion.py`) segments each RGB frame, deprojects the depth image into 3D points, fuses the semantic and geometric risk, and accumulates the result into a persistent BEV costmap published on `/risk_costmap` (Chapter 4).

2. The **MPPI node** (`mppi_node.cu`) subscribes to `/risk_costmap`, the robot pose, and `/goal_pose`. It uploads the costmap into a CUDA texture, runs $K = 5120$ GPU rollouts over a horizon of $T = 200$ steps, and publishes the optimal (v, ω) command (Chapter 5).

3. The **evaluation node** (`eval_run.py`) logs quantitative metrics for every goal-to-goal run (Chapter 6).

3.4 Coordinate Frames and Localisation

A subtle but decisive issue is that the risk map, the goals, and the controller's state must all live in one consistent frame. In the Gazebo worlds the robot spawns at a non-zero world pose while its wheel odometry starts at the origin, so the odometry frame is both *offset* from the world frame and *drifts* over time (up to ≈ 1 m after sustained rotations). Left uncorrected, this makes the controller reach a goal a metre away from the true target and makes the costmap bounds cover the wrong region.

The project anchors everything to a world-fixed frame using a `ground_truth_tf` node that publishes the transform world $\rightarrow$ odom from the simulator's ground-truth pose at 20 Hz. The perception node runs in the world frame, RViz's fixed frame is the world frame, and the MPPI node consumes the ground-truth pose so that its state, the costmap, and the goals are numerically consistent.

On localisation

Using the simulator's ground-truth pose stands in for a proper localisation system (e.g. AMCL or a SLAM stack). Providing metric self-localisation on the real robot is outside the scope of this internship; it is the natural companion to the perception and control contributions and is noted as future work.

CHAPTER 4

Vision-Based Terrain Risk Mapping

This chapter describes the perception pipeline that converts the RGB-D stream into a metric risk map. It is a **two-layer** architecture: a *semantic* layer for surface type and a *geometric* layer for rigid obstacles, fused per point and accumulated into a bird's-eye-view (BEV) costmap.

4.1 Hazard Catalogue and Risk Scores

The risk scores r_c follow the Week-1 hazard catalogue, ordered by increasing severity (Table 4.1). The scores are indicative and were calibrated on the platform during the project. The hierarchy actually used by the deployed model is tile $= 0$; small_step $= 0.35$; wet $= 0.5$; mud $= 0.85$; stairs $= 1.0$.

Table 4.1: Indoor urban hazard catalogue for the TurtleBot 4 (risk scores r_c).

Hazard type	Score r_c	Detectable by	Consequence
Flat dry floor (tile)	0.0	LiDAR + camera	None: ideal surface
Small step (≈15 mm)	0.35	Camera + IMU	Vibration, traversable but risky
Wet tile	0.50	Camera	Strong slip risk
Mud / thick carpet	0.85	Camera	Loss of traction, worst traversable
Staircase	1.00	LiDAR + camera	Irreversible fall, hardware damage

The critical case

Unlike a wet tile, which causes gradual degradation, a single stair step causes an *immediate and irreversible* fall. The 2D LiDAR sees only the first step edge, not the staircase pattern; the OAK-D camera identifies the texture and geometry of stairs at 3–5 m, giving MPPI time to plan an avoidance. The staircase class is therefore treated as a hard barrier, like a box (Section 5.2).

4.2 Dataset and Semantic Segmentation

Dataset. A custom dataset of ground images was collected in simulation and annotated (via Roboflow) into six classes (`background`, `carpet`, `small_step`, `stairs`, `tile`, `wet`) with dense per-pixel semantic masks. It contains 951 images in total, split into 890 (train) / 39 (validation) / 22 (test). Roboflow's augmentation pipeline (random horizontal flip, rotation $\pm 10^\circ$, shear, and brightness jitter) generated 10 variants per source frame to enlarge the training set.

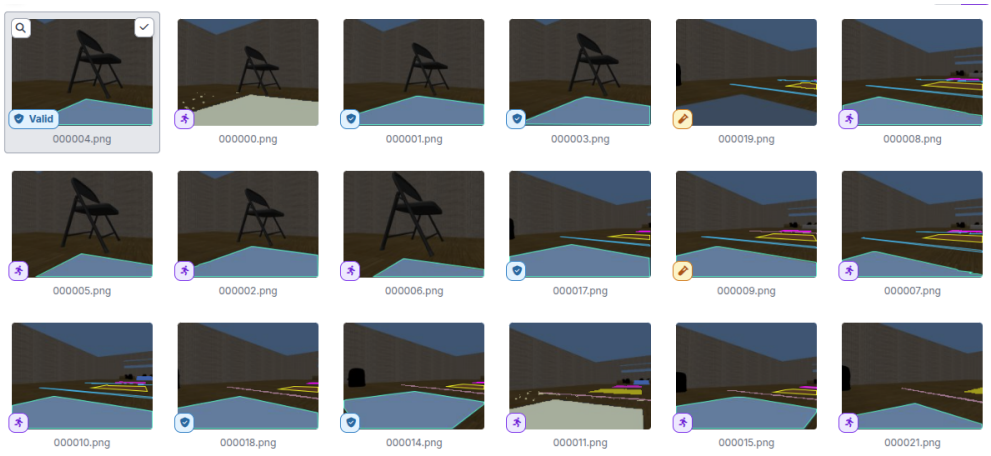


Figure 4.1: Dataset annotation in Roboflow: per-pixel semantic masks (hazard outlines) drawn on simulation renders of the `warehouse_risk` world.

Models compared. Three segmentation backbones were trained and benchmarked for the semantic layer, trading accuracy against embedded-inference speed (Table 4.2):

Table 4.2: Segmentation backbones evaluated for the semantic layer.

Model	Test mIoU	Role
SegFormer-B0	0.805	Reference / comparison
DeepLabV3+ / MobileNetV2	0.823	Embedded (Raspberry Pi, ONNX, ≈27% faster on CPU)
YOLO26 (`yolo26n-sem`)	≈0.80 (val 0.85)	**Deployed in the ROS nodes**

The deployed nodes use **YOLO26** at an input size of 320×320 (native 320×240), which is roughly $4\times$ faster on CPU than 640 while remaining accurate enough. The raw network forward produces class logits; the node computes $R_{\text{sem}}(\boldsymbol{p}) = \sum_c r_c \mathbb{P}(c \mid \boldsymbol{p})$ directly on the GPU via a softmax weighted by the risk vector r_c.

Figure 4.2 shows the semantic risk map produced by the deployed YOLO26 model from a test image (RGB | segmentation | risk | overlay); the reference SegFormer-B0 model yields a visually similar map, consistent with its comparable test mIoU (Table 4.2).

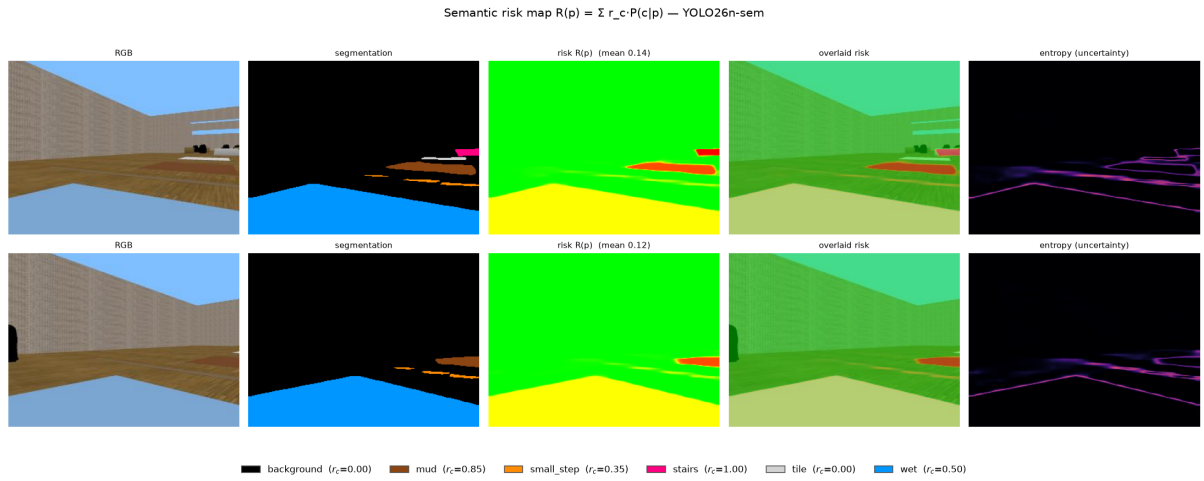


Figure 4.2: Semantic risk map (YOLO26, the model deployed in the ROS nodes): input RGB, predicted segmentation, expected risk $R(\boldsymbol{p}) = \sum_c r_c \mathbb{P}(c \mid \boldsymbol{p})$, and overlay (green = safe, red = danger).

4.3 The Two-Layer Risk Map

The semantic layer knows only the terrain *type*: `background` is generic safe floor ($r_c = 0$), *not* an obstacle. The risk of walls and rigid obstacles, which the camera labels `background` but which are clearly not drivable, comes from a separate **geometric** layer.

4.3.1 Geometric Layer

For each depth pixel, the node deprojects a 3D point using the camera intrinsics K, transforms it into a gravity-aligned world frame via TF, and reads its height h. Points above a small threshold are obstacles, with a linear ramp to full risk:

$$R_{\text{geo}}(h) = \text{clip}\left(\frac{h - h_{\text{obs}}}{h_{\text{full}} - h_{\text{obs}}}, 0, 1\right), \tag{4.1}$$

with h_{obs} set just above the small-step height so that a traversable small step stays governed by its *semantic* score while anything taller becomes a geometric obstacle.

4.3.2 Fusion

The two layers are fused by taking the *cautious maximum* per point:

$$R(\boldsymbol{p}) = \max\big(R_{\text{sem}}(\boldsymbol{p}), R_{\text{geo}}(\boldsymbol{p})\big). \tag{4.2}$$

A semantically safe but tall `background` point (a wall) takes the geometric risk; a wet area on the ground keeps its semantic risk. The result is published as a colored point cloud (`/risk_cloud`) and an overlay image (`/risk_image`) for visualisation.

4.4 BEV Costmap for Planning

MPPI needs a *stable, world-sized* cost field, not the instantaneous camera field of view. The node therefore accumulates the fused risk into a persistent top-down `OccupancyGrid` (`/risk_costmap`, resolution $0.10\,\text{m}/\text{cell}$) covering the whole world. Several design choices make this map usable by the controller:

- **Memory (max per cell).** Each cell keeps the *maximum* risk ever observed there, so a hazard stays on the map even when the camera looks away and its instantaneous view rotates past.
- **Time-based decay (half-life $\approx$180 s).** A cell not re-observed slowly loses value; genuine hazards are re-seen and stay near the max, while projection errors heal instead of poisoning the map forever. A fully-forgotten cell returns to *unknown* (-1), not to "safe".
- **Inflation ($\approx$0.2 m).** A circular `maximum_filter` dilates every hazard by the robot radius, because the controller samples risk at the robot's *centre*; a rollout grazing an obstacle by 5 cm would otherwise cost nothing.
- **Clearance halo.** An exponential cost gradient is added *around rigid obstacles only* ($R \geq 0.9$), so that moving away from a wall is always slightly cheaper, so the robot centres itself in passages instead of wall-hugging.
- **Rotation freeze.** When the pose TF is stale during a fast rotation, accumulation is frozen to avoid smearing walls into the room.

Optimism toward the unknown: an assumed design choice

Unknown cells (-1) are treated as *zero risk* by the controller. This makes the robot optimistic about unexplored ground: it will happily enter regions it has never seen. The alternative (treating the unknown as risky) tends to freeze the robot in place. This is an assumed trade-off, discussed again in Chapter 7.

The heavy operations (inflation, distance transform for the halo) are throttled to $\approx$2 Hz and cached, while the grid itself is republished at full rate, keeping `/risk_costmap` responsive at low CPU cost.

CHAPTER 5

Risk-Aware MPPI Controller

This chapter presents the controller that turns the risk map into motion. It is a GPU MPPI controller built on the MPPI-Generic library [4], with a custom cost function that samples the risk map from a CUDA texture. A single parameter, λ_{risk}, moves the robot from risk-blind (the $\lambda_{\text{risk}} = 0$ baseline) to strongly risk-averse.

5.1 Dynamics and Control

The robot is modelled as a planar unicycle (a Dubins car), with state $\boldsymbol{x} = (x, y, \theta)$ and control $\boldsymbol{u} = (v, \omega)$:

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega. \tag{5.1}$$

The controls are bounded to the TurtleBot 4 limits: $v \in [0,\, 0.30]\,\mathrm{m\,s^{-1}}$ and $\omega \in [-1.5,\, 1.5]\,\mathrm{rad\,s^{-1}}$.

No reverse motion ($v \geq 0$)

The lower bound $v \geq 0$ is deliberate: the camera does not see behind the robot, so the risk there is unknown. A consequence is that the only way out of a risk cul-de-sac is to rotate on the spot, which matters when analysing the robot's behaviour in tight scenarios (Chapter 7).

5.2 The Risk-Aware Cost Function

For each rolled-out state the cost combines an attraction to the goal, the terrain risk, and a hard barrier for lethal cells:

Per-rollout cost

$$S_k \;=\; \sum_{t=0}^{T-1} \Big[\, w_{\text{goal}}\, d_t \;+\; \lambda_{\text{risk}}\, R(x_t, y_t) \;+\; \mathbb{1}[\text{crash}] \cdot c_{\text{crash}} \,\Big] \;+\; w_{\text{term}}\, w_{\text{goal}}\, d_T$$

where $d_t = \|(x_t, y_t) - \text{goal}\|$ is the (Euclidean) distance to the goal, $R \in [0, 1]$ is the risk sampled from the texture, and a cell with $R >$ crash_threshold ($= 0.95$, i.e. staircases) triggers a persistent penalty c_{crash}.

Two design points matter:

- **Linear (not squared) distance.** Using d_t rather than d_t^2 keeps the goal-attraction gradient *constant* with distance. With d_t^2, far from the goal the distance term dwarfs the risk term (the robot ploughs through hazards), while near the goal the opposite happens. Linear distance makes the $w_{\text{goal}}/\lambda_{\text{risk}}$ balance hold at *every* distance.
- **Risk is a toll, not a wall.** Except for the crash barrier (reserved for staircases), risk is a *price* the controller is willing to pay if no cheaper detour exists. This is the essence of the arbitrage analysed in Section 5.5.2.

5.3 Sampling and Exploration

Rollouts are generated with **colored (pink) noise**, i.e. temporally correlated perturbations (spectral exponent 1). At equal variance, pink noise concentrates energy at low frequency, so rollouts *drift* coherently left or right instead of vibrating around the nominal, which is exactly what is needed to discover a detour such as "go around to the north". White noise at the same standard deviation explores narrower tubes. The default standard deviations are $\sigma_v = 0.15$ and $\sigma_\omega = 0.70$.

5.4 GPU Implementation

The controller is a `VanillaMPPIController` templated on the Dubins dynamics, the custom `RiskMapCost`, a DDP feedback controller, and a colored-noise sampler. The configuration is $T = 200$ steps and $K = 5120$ rollouts. An initial 20 Hz loop ($dt = 0.05\,\mathrm{s}$, horizon 5 s at the original $T = 100$) was found unnecessarily fast for this platform: the TurtleBot 4's kinematics are slow ($v_{\max} = 0.30\,\mathrm{m\,s^{-1}}$, $\omega_{\max} = 1.5\,\mathrm{rad\,s^{-1}}$), so a shorter horizon buys little reactivity while wasting the horizon's reach. The retained configuration, used throughout the Chapter 7 campaign, is 14 Hz ($dt \approx 0.0714\,\mathrm{s}$) with $T = 200$, giving a horizon of $T/\text{control\_hz} \approx 14.3\,\mathrm{s}$ and a look-ahead reach of $T \times v_{\max} \times dt \approx 4.3\,\mathrm{m}$. On the development workstation (RTX 5070 Laptop) a solve takes $\approx 3\,\mathrm{ms}$, well under the 71 ms cycle budget at 14 Hz.

The risk map is uploaded into a 2D CUDA *texture* (`TwoDTextureHelper`), which provides free hardware bilinear interpolation and edge clamping when the kernel queries $R(x,y)$ at arbitrary continuous coordinates. The `OccupancyGrid` integers $[-1, 100]$ are mapped to $[0,1]$ floats, with unknown (-1) mapped to 0 (the optimism of Section 4.4).

Horizon length matters

An early 3 s horizon was too short: crossing a 1 m risk patch took longer than the horizon, so no rollout ever "saw" the exit and the controller faced an unsolvable detour-vs-cross dilemma. At 14.3 s (the retained 14 Hz, $T = 200$ configuration) the full crossing, and the detour around it, comfortably fits inside the horizon, and the arbitrage of Section 5.5.2 becomes well-posed. Because T is a compile-time template constant, the horizon in seconds equals $T/\text{control\_hz}$; the loop rate is the single knob that scales it, at the cost of a coarser dt (each step advances $\approx 2.1\,\mathrm{cm}$).

5.5 Tuning: What the Controller Actually "Sees"

A key deliverable of the internship is a *quantitative* tuning model that predicts behaviour from the cost formula, rather than tuning by trial and error. The units of the cost (illustrated here with $w_{\text{goal}} = 4$, $w_{\text{term}} = 10$, and $v_{\max} = 0.30$, at the retained 14 Hz, $T = 200$ configuration; the evaluation runs of Chapter 7 use the higher $w_{\text{goal}} = 10$, see the warnbox below) give three conversions:

- 1 m **of a zone** ≈ 47 **steps.** At $v_{\max}$ and 14 Hz, one step advances $\approx 2.1\,\mathrm{cm}$. Crossing 1 m of wet ($R = 0.5$) costs $\lambda_{\text{risk}} \times 0.5 \times 47 \approx 23.3\,\lambda_{\text{risk}}$.
- **The horizon "sees"** $\approx 4.3\,\mathrm{m}$**.** 200 steps $\times$ $\approx 2.1\,\mathrm{cm}$; anything beyond that of path exists for the controller only through the terminal term.
- **Being** 1 m **farther from the goal costs** 840 **per cycle** ($200 \times w_{\text{goal}} + w_{\text{term}} w_{\text{goal}}$), the "goal pressure" that opposes risk. This ratio is set by w_{goal}, w_{term}, and T alone, so it does not depend on the loop rate itself, only on the value of T chosen.

5.5.1 Risk–distance equivalence

At $\lambda_{\text{risk}} = 15$, a wet cell costs 7.5 per step, as much as being 1.9 m farther from the goal for that step ($7.5/w_{\text{goal}}$). **MPPI does not see a forbidden zone; it sees terrain where each centimetre costs ≈ 1.9 m of detour.** If no detour is cheaper than that, it crosses. This is a negotiation, never a wall: only `crash_threshold` builds a true wall.

5.5.2 The crossing-vs-detour threshold

For a detourable patch (robot at the edge, detour within the horizon), there is a λ_{risk} threshold $\lambda^{\star}_{\text{risk}}$ above which detouring wins and below which crossing wins (Table 5.1).

Table 5.1: Crossing-vs-detour thresholds for a detourable 1 m *patch.*

Patch risk	$\lambda^{\star}_{\text{risk}}$
0.35 (small step)	≈ 17
0.50 (wet)	≈ 12
0.85 (mud)	≈ 7

$\lambda_{\text{risk}} = 15$ is the "intelligent" setting

At $\lambda_{\text{risk}} = 15$ the thresholds fall so that the robot *crosses* the mild small step when it is shorter, but *detours* around wet and mud. Below ≈ 5 it crosses everything; above ≈ 30 it detours everything and refuses forced barriers. The class hierarchy r_c only produces distinct behaviour when λ_{risk} places the thresholds *between* the class values: that, not "bigger is safer", is the real criterion for choosing λ_{risk}.

$\lambda^{\star}_{\text{risk}}$ scales with w_{goal}

Table 5.1 is computed at $w_{\text{goal}} = 4$. Because the threshold balances a fixed risk cost against a goal-distance cost, raising w_{goal} raises $\lambda^{\star}_{\text{risk}}$ roughly proportionally: some of the evaluation runs in Chapter 7 use $w_{\text{goal}} = 10$, for which the wet threshold moves from ≈ 12 to ≈ 40–54 (recomputed for the exact evaluation geometry in Section 7.3). The qualitative reasoning is unchanged; only the numeric threshold shifts.

Two structural effects, predicted by the same cost arithmetic, explain the behaviour observed near a forced (wall-to-wall) barrier:

- **Hesitation window.** Between ≈ 1 m and the patch edge, "wait" is briefly cheaper than crossing (the risk is ahead, the progress gain does not yet compensate). Its width grows with λ_{risk}; this is the slowing/floating seen in front of hazards, and it is structural, not a bug.
- **Inflation ratchet.** At the inflated edge, staying put pays R for the full $T = 200$ steps of the horizon, whereas crossing the 1 m patch pays it for only ≈ 47 steps; crossing becomes optimal again, and each further centimetre inward reduces the remaining toll, hence the observed "hesitate, then commit" pattern.

5.5.3 The softmax temperature λ_{temp} and the ESS

λ_{temp} controls only the *sharpness of the vote* among rollouts. It must be compared to the cost *gaps* between rollouts on the scale the softmax actually sees, and here a subtle implementation detail matters:

the MPPI-Generic kernel divides the total cost by T, so the softmax sees the *mean cost per step* (of order ~ 30, not ~ 3000). Gaps within a route family are ~ 0.05–0.5; gaps between families (cross vs detour) are ~ 1–20.

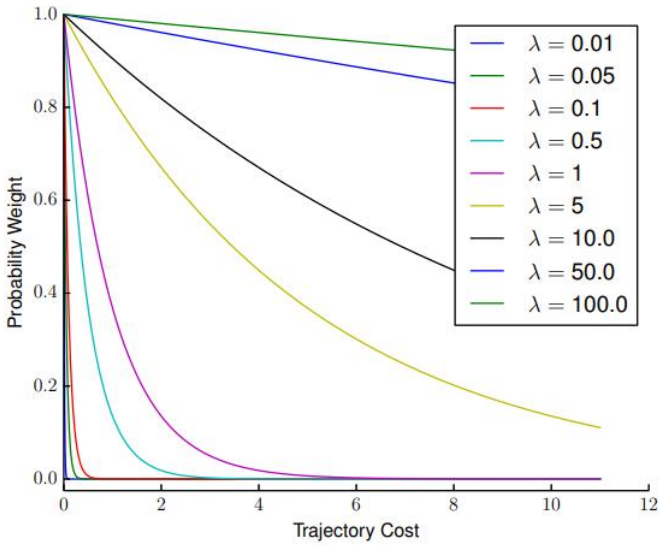


Figure 5.1: Softmax weight $w_k \propto \exp(-(S_k - S_{\min})/\lambda_{\text{temp}})$ as a function of a rollout's cost gap to the best one, for several λ_{temp}. Small λ_{temp} (≤ 0.1) collapses almost immediately to zero weight: near-argmax, i.e. low ESS; large λ_{temp} (≥ 10) stays close to 1 across the whole cost range: every rollout counts almost equally, i.e. high ESS (the "mush" pathology at $\lambda_{\text{temp}} = 5$–$10$ described below).

Drive λ_{temp} by the effective sample size

The right dial is the *effective sample size* (ESS $= \eta/N$, published live on `/mppi_diag`). Aim for ESS $\approx$ 1–5% of N. Too low (ESS $\to$ 1 rollout) gives near-*argmax* behaviour: twitchy commands, possible flip-flop between tied families (zigzag). Too high (ESS $\to$ 50%+) averages "left+right+stop+forward" into mush, and the robot circles slowly and makes no progress. Measured on this code, the old default $\lambda_{\text{temp}} = 5.0$ gave ESS $\approx 50\%$ (pathological); the working default is $\lambda_{\text{temp}} \in [0.1, 1.0]$.

5.5.4 Cost-shaping ($\alpha = 1$)

The node runs with $\alpha = 1$, which cancels the control-cost term of the importance-sampling weight (Section 2.2): a pure cost-shaping MPPI whose smoothness comes from the pink noise. Exposing α enables an ablation ($\alpha \in \{1.0, 0.9, 0.5\}$): $\alpha < 1$ penalises deviation from the nominal, adding inertia (less zigzag) at the price of slower avoidance.

5.6 Live Diagnostics

To make the controller legible, the node publishes at every cycle:

- `/mppi_rollouts`: a sample of ≈ 64 rollouts, colored from green (cheapest) to red (most expensive): the controller's "inner vision". The competing route families and the hesitation window become directly visible in RViz.

- `/mppi_diag`: a vector [baseline, free-energy mean, ESS%, solve_ms, plan risk sum, plan goal cost, plan max risk]. `plan risk sum` is the integrated risk of the chosen plan *before* execution, comparable to the ground-truth metric of Chapter 6.

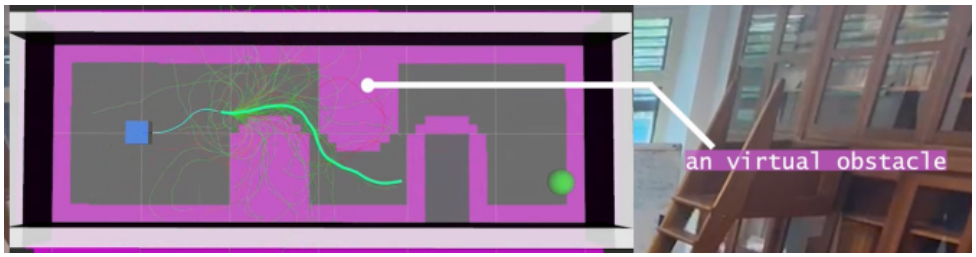


Figure 5.2: RViz view of the sampled rollout cloud (thin green/grey traces) and the chosen plan (thick green line, from the blue start marker to the green goal) bending around a virtual obstacle (magenta) baked into the costmap. The competing route families explored by the controller are directly visible.

CHAPTER 6

Experimental Setup and Scenarios

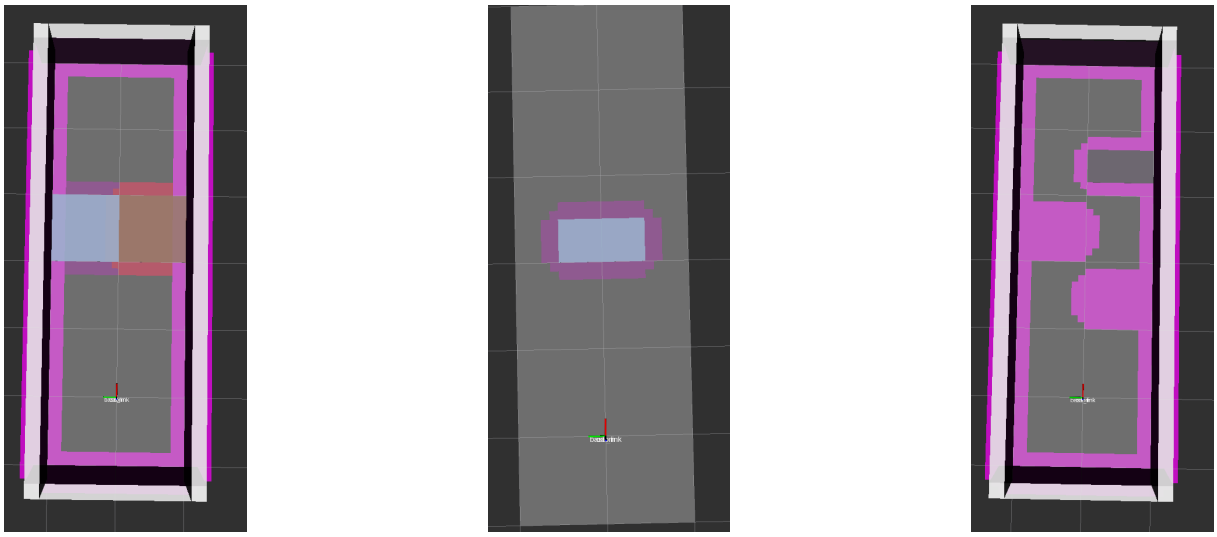
This chapter describes the scenarios and metrics used to evaluate the system on top of the simulation environment already introduced in Section 3.2: the two evaluation modes (perception-in-the-loop, controller-in-isolation) and the demonstration on the physical robot.

6.1 Controlled Scenarios

The scene geometry can be *baked* directly into `/risk_costmap` by the `virtual_scenario` node (no Gazebo, no camera, no perception), together with matching RViz markers. Because the geometry is known ground truth, this isolates the *controller* from perception noise. Three layouts are demonstrated in this report (Table 6.1), each isolating one decision. The robot may also be virtually "re-started" elsewhere in the scene without touching the zone definitions.

Table 6.1: Baked virtual scenario layouts demonstrated in this report (`virtual_scenario`).

Layout	What it tests
`barrier`	A risk-zone belt spanning the corridor: cross through the cheaper (wet) half or pay full price in the lethal half.
`zigzag`	Slalom of lethal obstacles, weave left/right/left.
`cross_or_detour`	A lone wet patch in an *open* field: cross (short) or detour (long), the crossing-willingness test.



(a) `barrier` *(b)* `cross_or_detour` *(c)* `zigzag`

Figure 6.1: RViz top-down view of the three demonstrated baked layouts (magenta = lethal/wall, blue-grey = wet patch, brown = mud): a forced wall-to-wall crossing, an isolated patch with a free detour, and a slalom of obstacles.

6.2 Evaluation Methodology

The `eval_run` node logs one record per goal-to-goal run (to `runs.jsonl`). A run starts on each `/goal_pose` and ends when the ground-truth pose reaches the goal tolerance, or on timeout. The metrics are:

- **Ground-truth risk integral** $\int r_{\text{gt}}(x,y)\,dt$: the *true* risk exposure, computed from the analytically-known world zones, so it is comparable across runs regardless of what perception saw. This is the primary safety metric.
- **Path length**, **duration**, **mean speed**.
- **Time spent in each hazard zone**.
- **Perceived risk** (max and integral, sampled from `/risk_costmap`).
- The controller parameters (λ_{risk}, λ_{temp}, weights, …), archived with every run for ablation.

The central comparison

The headline experiment is **standard MPPI ($\lambda_{\text{risk}} = 0$) vs. risk-aware MPPI ($\lambda_{\text{risk}} > 0$)** on the same scenario and goal. A safe controller should reduce the ground-truth risk integral, ideally at a modest cost in path length and time. Sweeping $\lambda_{\text{risk}} \in \{0, 5, 10, 15, 25, 50\}$ traces the full safety-vs-efficiency trade-off.

Protocol. One variable at a time, several runs per point, fixed goal. Beyond the central λ_{risk} sweep, secondary studies vary λ_{temp} (read the ESS), σ_ω, white vs. pink noise, and α (Section 5.5). Each study maps to one results sub-section, with the theoretical prediction placed next to the measurement.

6.3 Evaluation Modes and Real-Robot Demonstration

The controller reads the risk field from exactly one place, the `/risk_costmap` topic, regardless of who produces it. This single interface makes two *producers* interchangeable and defines the two evaluation modes used in this work.

One interface, two producers

Perception and control are validated *separately*, then shown to compose over the same `/risk_costmap` contract:

- **Mode A (perception in the loop):** the costmap is produced by the vision pipeline (camera → segmentation → R_{sem} → depth geometric layer → fusion → BEV, Chapter 4) in a Gazebo hazard world. Validates the *full* pipeline end to end.
- **Mode B (controller in isolation):** the costmap is a *ground-truth* map baked directly from known scene geometry by `virtual_scenario` (no camera, no vision). Isolates the *controller* with a perfect, perfectly repeatable map.

6.3.1 Mode A: perception in the loop (simulation)

Launched by `scenario.launch.py` (cage) or `risk_world.launch.py` (warehouse): a simulated TurtleBot 4 drives while its OAK-D camera feeds `risk_fusion`, which builds `/risk_costmap` from *what the camera actually sees*. This is the mode that exercises the vision contribution: `/risk_cloud` (colored point cloud), `/risk_image` (heatmap overlay on the RGB), and the costmap accumulating from the camera field of view. It is heavier (GPU camera rendering) but is the only mode that demonstrates perception → control.

6.3.2 Mode B: controller on a ground-truth map (simulation and real robot)

Here `virtual_scenario` bakes the chosen layout (Table 6.1) directly into `/risk_costmap`. Because the map is exact and instantaneous, this mode removes all perception error and isolates the controller. It runs in two configurations:

- **Simulation** (`virtual_scenario_sim.launch.py`): a minimal diff-drive robot in an empty Gazebo world executes the commands, so the robot can physically cross a virtual wall if MPPI fails to avoid it (the observable failure mode). Used for the clean behavioural experiments and parameter sweeps across the three layouts.
- **Real robot** (`virtual_scenario_real.launch.py`): the physical TurtleBot 4 executes the commands. **The real robot moves through the room while avoiding obstacles and risk zones that exist only in the costmap and in RViz.** This is the hardware demonstration.

Why evaluate on a ground-truth map: a deliberate choice

A physical hazard course (real mud, wet floor, and above all *staircases*) is costly, hard to make repeatable, and dangerous for the robot: a real stair step means an irreversible fall and hardware damage. Evaluating on a controlled, ground-truth risk field instead gives *exact* risk-exposure measurements (no perception noise to confound them), perfect *repeatability*, *safety* for the hardware, and the ability to *sweep parameters* systematically. The lighter compute of Mode B is a further advantage: more runs, faster iteration.

Real-robot networking caveat

The real robot is reachable only through a Fast-DDS discovery server: every process, the launch and the goal publisher, must be started from the same sourced environment, or the ROS graph splits in two and the controller never receives odometry or goals. This is an operational caveat, not a limitation of the method.

A Mode A capture showing the camera-built heatmap and costmap is given later alongside the baseline-vs-risk-aware comparison (Figures 7.6–7.7, Chapter 7). Figure 6.2 shows the Mode B hardware demonstration itself: the physical TurtleBot 4, with its onboard laptop running the controller, navigating a room in which the only trace of the hazard is the RViz overlay: no camera, no obstacle, nothing a bystander could see.

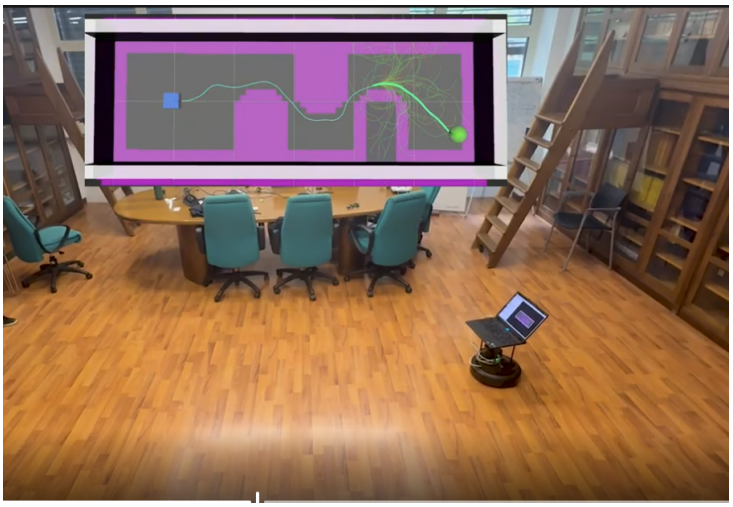


Figure 6.2: The real robot demonstration: the physical TurtleBot 4 (foreground) navigates the `zigzag` layout, composited with the corresponding RViz view (top) showing the baked virtual obstacles (magenta), the rollout cloud, and the chosen path (green), none of which exist in the physical room.

6.3.3 Attempted: on-board perception on the real robot

The natural next step, the real robot driven by its *own* camera perception, was attempted with a "ghost-twin" architecture, following the supervisor's suggestion that *"the real robot physically moves on the actual terrain, while a simulated counterpart replicates the same trajectory and generates the camera observations in a controlled simulated environment."* A Gazebo twin, teleported every cycle onto the real robot's pose, would produce the RGB-D stream that `risk_fusion` turns into the costmap, while MPPI commands the real robot.

Basic point-to-point control of the real robot under MPPI *succeeded*, but coupling it to the twin's perception did *not*: reliably spawning the twin and streaming its camera, together with the Fast-DDS communication between the real robot and the simulated perception, proved unreliable within the internship. This integration is therefore left as future work (Chapter 9). Importantly, it does not affect the two building blocks, which were each validated independently: the *perception* in Mode A and the *controller* in Mode B.

CHAPTER 7

Results and Discussion

7.1 Perception Results

The three segmentation backbones reached test mIoU of 0.805 (SegFormer-B0), 0.823 (DeepLabV3+/MobileNetV2), and ≈ 0.80 (YOLO26), the last being the deployed model at 320×320 input. Qualitatively, the risk map of Figure 4.2 shows clean separation of hazard classes. The per-class agreement and the on-CPU inference speed of the three backbones are compared in Figures 7.1 and 7.2. All three reach a comparable mean IoU; YOLO26 is retained for deployment because it is 5–6$\times$ faster on CPU (about 46 vs 7–10 FPS), which enables GPU-free on-board inference.

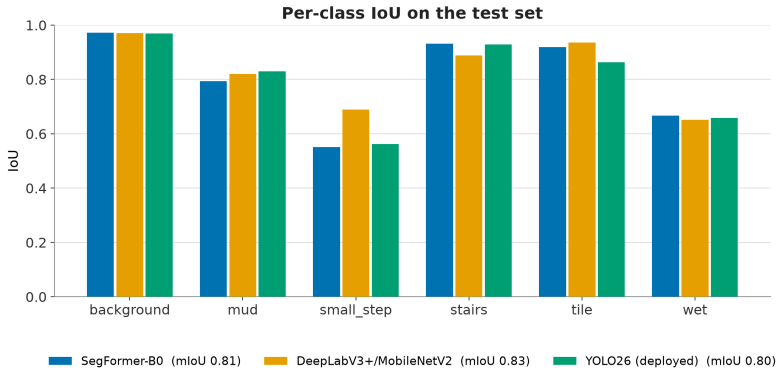


Figure 7.1: Per-class IoU on the test set for the three trained segmentation models (SegFormer-B0, DeepLabV3+/MobileNetV2, and the deployed YOLO26).

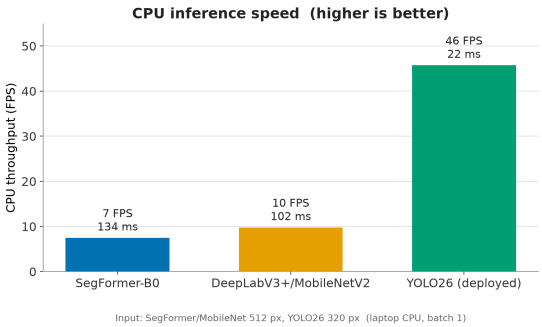


Figure 7.2: CPU inference speed of the three models (higher is better); the deployed YOLO26 is markedly faster, enabling embedded use.

7.2 Baseline vs. Risk-Aware MPPI

The central result contrasts $\lambda_{\text{risk}} = 0$ (risk-blind baseline) with the tuned risk-aware controller on the same scenario and goal. Table 7.1 is the λ_{risk} sweep; the expectation from Chapter 5 is that the ground-truth risk

integral falls as λ_{risk} rises, at a moderate cost in path length, until λ_{risk} grows so large that the hesitation window causes stalling.

Table 7.1: λ_{risk} sweep (scenario `cross_or_detour`, goal x=4.5 m, single run per row).

λ_{risk}	**Risk $\int r_{\text{gt}} dt$**	**Path (m)**	**Time (s)**	**Behaviour**
0 (baseline)	1.02	4.30	18.1	crosses
5	1.00	4.30	18.0	crosses
10	0.99	4.30	18.0	crosses
15	0.99	4.29	17.9	crosses
25	0.97	4.30	17.9	crosses
50	0.00	4.70	19.7	detours

Up to $\lambda_{\text{risk}} = 25$, the controller consistently crosses the wet patch: the ground-truth risk integral is essentially flat (≈ 1.0, the unavoidable cost of the ≈ 2 s spent inside the R=0.5 zone) and both path length and time are unchanged from the baseline: crossing is essentially free at this scale, so no λ_{risk} in this range makes detouring worth it. By $\lambda_{\text{risk}} = 50$ the behaviour has flipped completely: the risk integral drops to **zero** (the patch is avoided entirely) at a cost of only +9% path length and +9% time. The transition itself is not sharp, as Figure 7.3 and Section 7.3 detail: it starts around $\lambda_{\text{risk}} \approx 28$ and is stochastic (pink-noise dependent) before settling into a reliable detour by $\lambda_{\text{risk}} \approx 38$.

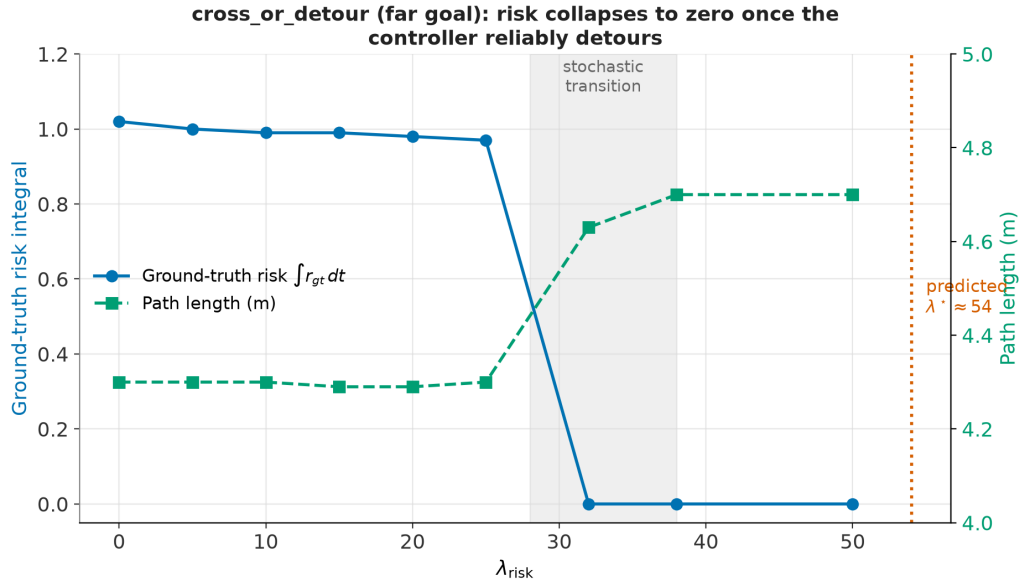


Figure 7.3: Ground-truth risk integral (left axis) and path length (right axis) vs. λ_{risk} on `cross_or_detour` (goal x=4.5). The risk integral stays flat while the controller crosses, then drops to zero once it reliably detours; the predicted threshold $\lambda^{\star}_{\text{risk}} \approx 54$ (Section 7.3) is marked alongside the measured transition band ($\approx$ 28–38).

A second, forced-crossing comparison confirms the same trend on the `barrier` layout (wall-to-wall belt, wet north / mud south) and, pushed further, reveals a second effect the theory predicts but the `cross_or_detour` sweep alone cannot show: a full extended sweep ($\lambda_{\text{risk}} \in \{0, 5, 10, 15, 25, 27, 35, 38, 38.5, 44\}$, plus an observation at $\lambda_{\text{risk}} = 50$) is reported in Table 7.2 and Figure 7.4.

With $\lambda_{\text{risk}} = 0$ the controller has no reason to prefer either half and crosses through the costlier mud; from $\lambda_{\text{risk}} = 5$ up to at least 27 it consistently and quickly takes the cheaper wet half instead, cutting the ground-truth risk integral by **44%** (from 3.43 to 1.93) at a cost of only +2–6% path length and time: safety improves sharply while efficiency is essentially preserved.

Table 7.2: Extended λ_{risk} sweep on the `barrier` layout (forced crossing, same start/goal, single run per row; $\lambda_{\text{risk}} = 50$ never reached the goal within the 240 s timeout and is reported separately).

λ_{risk}	**Risk** $\int r_{\text{gt}} dt$	**Path (m)**	**Time (s)**	**Behaviour**
0	3.43	4.30	18.0	crosses mud (south, R=0.85)
5	1.97	4.31	17.9	crosses wet, clean
10	1.98	4.36	18.2	crosses wet, clean
15	1.98	4.35	18.1	crosses wet, clean
25	2.00	4.44	19.0	crosses wet, clean
27	1.93	4.38	18.6	crosses wet, clean
35	1.95	5.21	27.9	crosses wet, mild hesitation
38	1.94	11.76	96.5	crosses wet, long erratic loop
38.5	1.93	4.40	19.6	crosses wet, clean (same λ_{risk}, opposite outcome)
44	1.90	24.89	234.7	crosses wet, near-timeout
50	n/a	n/a	> 240 (cut off)	**never crosses**: locks into a tight loop in front of the belt

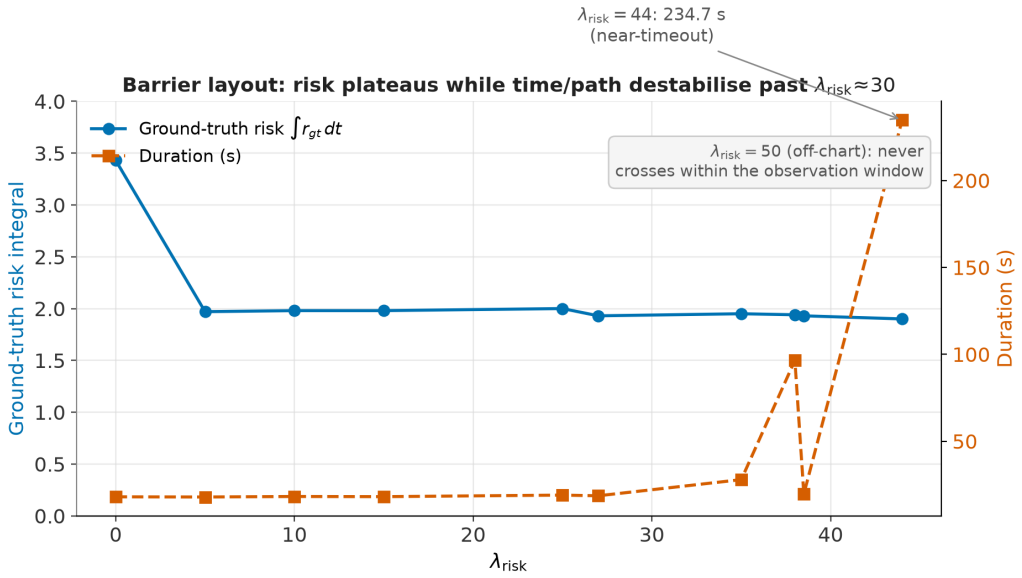


Figure 7.4: Ground-truth risk integral (left axis) and run duration (right axis) vs. λ_{risk} on the `barrier` layout. The risk integral plateaus almost immediately (all crossings past λ_{risk}=5 take the wet half), while the duration stays flat up to $\lambda_{\text{risk}} \approx 30$ and then destabilises sharply, including two runs at nearly identical λ_{risk} (38 and 38.5) with an 80 s gap in outcome.

The hesitation window has a cost, even when it does not stall

Beyond $\lambda_{\text{risk}} \approx 30$–$35$, the ground-truth risk integral stops improving at all (it stays flat at ≈ 1.9–2.0) while the time and path length degrade sharply and *unpredictably*: two runs at $\lambda_{\text{risk}} = 38$ and 38.5, differing by under 2%, took 96.5 s and 19.6 s respectively. This matches the hesitation-window mechanism of Section 5.5.2: past a point, raising λ_{risk} buys no additional safety, only a widening, and increasingly stochastic, window in which the sampled rollouts do not agree on when to commit to the crossing. At $\lambda_{\text{risk}} = 50$ the window is wide enough that no rollout ever commits within the observation window: the robot circles in front of the belt indefinitely (Figure 7.5). **There is an optimal range, not a "more caution is always better" rule**: for this layout, $\lambda_{\text{risk}} \in [5, 27]$ already captures the full safety benefit at essentially no efficiency cost, and pushing further only adds risk of never arriving at all.

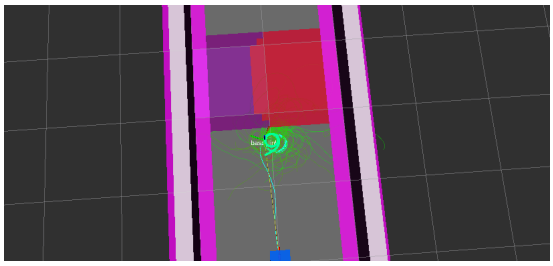


Figure 7.5: RViz view of the $\lambda_{\text{risk}} = 50$ lock-up: the rollout cloud (green) spirals in place immediately in front of the belt, never committing to a crossing family.

7.3 Validating the Cost-Arbitrage Model

The `cross_or_detour` scenario (a single wet patch, $R = 0.5$, in an open field) measures the crossing-to-detour switch directly. Two goals were tested at the actual evaluation settings ($w_{\text{goal}} = 10$, control_hz $=$ 14 Hz, $T = 200$): a *far* goal (x=4.5 m) and a *near* goal (x=3.0 m, just past the patch).

Recomputing $\lambda^{\star}_{\text{risk}}$ for this geometry. Table 5.1 was derived for a generic 1×3 m band at $w_{\text{goal}} = 4$ and gives $\lambda^{\star}_{\text{risk}} \approx 12$ for $R = 0.5$, but the evaluation runs use $w_{\text{goal}} = 10$ and the `cross_or_detour` patch has a different footprint (0.5 m wide, no walls). Numerically integrating the exact cost formula (Section 5.2) over idealised constant-speed paths for this specific geometry gives the predictions in Table 7.3.

Table 7.3: Predicted vs. measured crossing/detour threshold on `cross_or_detour` ($R = 0.5$, $w_{goal} = 10$).

Goal	Predicted $\lambda^{\star}_{\text{risk}}$	Measured	Observation
Far (x=4.5)	≈ 54	≈ 25–32	stable cross up to 25; unstable at 32; reliable detour by 38
Near (x=3.0)	≈ 40	≈ 32–34	stable cross up to 32; reliable detour by 34

Order of magnitude confirmed, exact value overestimated

The corrected model, at the right w_{goal}, moves the prediction from an order of magnitude too low (12, computed at $w_{\text{goal}} = 4$) to the right ballpark (40–54 at $w_{\text{goal}} = 10$), confirming that $\lambda^{\star}_{\text{risk}} \propto w_{\text{goal}}$ was the dominant missing factor. The idealised model still overestimates the measured threshold by roughly 40–60%: it assumes a perfectly executed straight-line crossing and a perfectly minimal

detour, whereas the real controller votes stochastically over pink-noise rollouts, so the hesitation-window mechanism (Section 5.5.2) lets "detour" win somewhat earlier than the deterministic geometric argument predicts.

A counter-intuitive detail: near beats far

Both the idealised model and the naive intuition ("a detour matters less when the total trip is long") predict a *lower* threshold for the near goal than the far one (40 vs 54), and indeed a shorter trip should make the detour's relative cost more visible. The measurement shows the opposite ranking: the near goal's threshold (≈ 33) is slightly *higher* than the far goal's (≈ 28). The likely explanation is horizon truncation: at $v_{\max} = 0.30\,\mathrm{m\,s^{-1}}$, $T = 200$ steps and control_hz $= 14\,\mathrm{Hz}$, the horizon reaches $T \cdot dt \cdot v_{\max} \approx 4.29\,\mathrm{m}$, *shorter* than the far goal (4.5 m) but comfortably longer than the near one (3.0 m). For the far goal, neither path reaches the goal within the horizon, so the terminal cost (evaluated at a point still short of the goal) enters the comparison differently than the simple full-path argument assumes. This is a genuine limitation of the idealised threshold model, not a measurement artefact: the real controller's finite horizon interacts with the goal distance in a way the geometric argument alone does not capture.

7.4 Secondary Studies

7.4.1 Softmax temperature λ_{temp} and the effective sample size

Section 5.5 predicts that the softmax temperature λ_{temp} should be driven by the effective sample size (ESS), targeting 1–5% of $N = 5120$ rollouts, and that the previous default $\lambda_{\text{temp}} = 5.0$ is pathologically high. Three runs on `cross_or_detour` ($\lambda_{\text{risk}} = 32$, otherwise identical settings) at $\lambda_{\text{temp}} \in \{0.2, 0.85, 5.0\}$ confirm this directly by reading the live ESS from `/mppi_diag` (Table 7.4).

Table 7.4: Effective sample size vs. softmax temperature λ_{temp} (`cross_or_detour`, $\lambda_{\text{risk}} = 32$, $N = 5120$ rollouts; ESS % measured over the stabilised tail of each run).

λ_{temp}	ESS (% of N)	Behaviour
0.2	$\approx 0.3\%$	near-argmax: far below target, twitchy vote
0.85	$\approx 5.1\%$	inside the 1–5% target band
5.0	≈ 24–28%	well above target: the softmax averages most rollouts together

The measurement confirms the qualitative prediction of Section 5.6: $\lambda_{\text{temp}} = 0.2$ collapses the vote onto a near-single rollout (ESS an order of magnitude below the target), while $\lambda_{\text{temp}} = 5.0$ pushes the ESS well past the pathological "mush" threshold, consistent with the old default having been changed. The working default $\lambda_{\text{temp}} = 0.85$ lands inside the recommended 1–5% band. The measured ESS at $\lambda_{\text{temp}} = 5.0$ ($\approx 25\%$) is lower than the $\approx 50\%$ figure quoted from earlier informal observation (Section 5.6), most likely because the cost landscape (and therefore the spread of rollout costs that the softmax has to average over) differs between scenarios; the qualitative conclusion, that $\lambda_{\text{temp}} = 5.0$ is far too high, is unaffected.

Scope of the secondary studies

Time constraints during the internship limited this section to the λ_{temp}/ESS study above. The pink-vs-white noise detour-discovery rate and the cost-shaping ablation ($\alpha \in \{1.0, 0.9, 0.5\}$, Section 5.5) were designed but not run; they are left as immediate future work (Chapter 9).

7.5 Qualitative Demonstrations

The two evaluation modes (Section 6.3) were both exercised on video. Mode A (perception in the loop) runs in the `warehouse_risk` Gazebo world; Mode B (controller on a ground-truth map) runs on the baked layouts of Table 6.1, in simulation and on the physical robot.

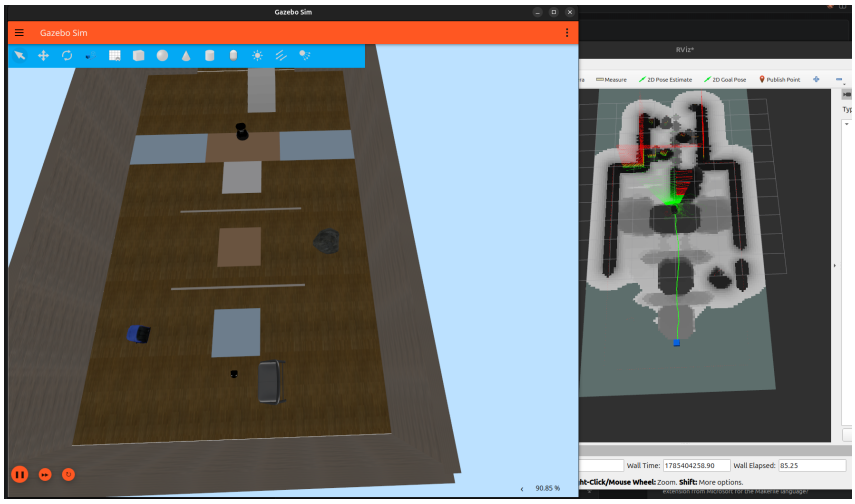


Figure 7.6: Mode A, risk-blind baseline ($\lambda_{\text{risk}} = 0$): Gazebo (left) and RViz (right, with the perception-built costmap and the robot's path in green). With no risk penalty the controller drives the shortest path straight through the wet/mud patches in front of it.

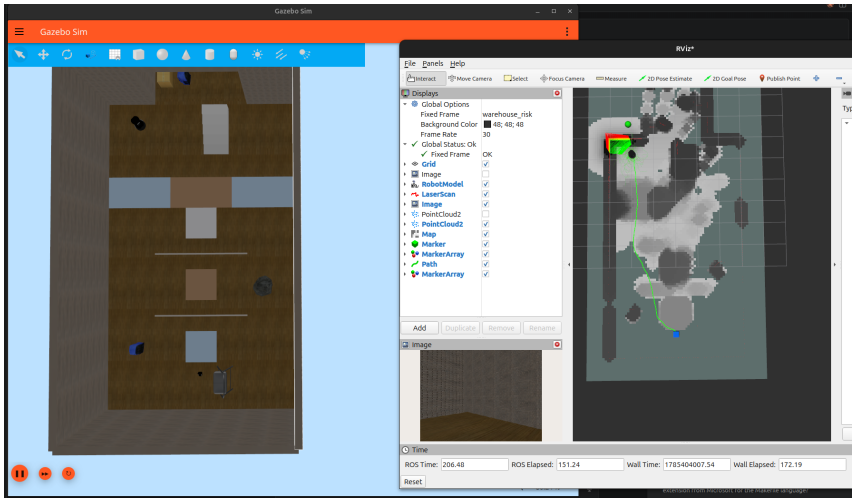


Figure 7.7: Mode A, risk-aware controller ($\lambda_{\text{risk}} = 27$), same world as Figure 7.6 (the goal is the green ball). The path (green, RViz) now visibly bends around the hazard patches instead of cutting through them: the qualitative counterpart of the λ_{risk} sweep of Section 7.2.

Figure 6.1 (Section 6.1) shows the ground-truth-map (Mode B) layouts from the same video set: the robot was recorded crossing the cheaper half of the `barrier` belt, taking the free detour in `cross_or_detour`, and weaving through the `zigzag` slalom, in both simulation and on the real robot navigating a purely virtual risk field. Video recordings of these three demonstrations, including the real-robot run, are available at: `https://drive.google.com/drive/folders/1MfgfFHclGiub44NgCKI6jODGo3IOm5lC?usp=sharing`.

7.6 Discussion and Limitations

Several honest limitations frame the results:

- **Localisation.** The controller relies on a ground-truth pose in simulation and on wheel odometry (which drifts) on the real robot; a metric localisation system (AMCL/SLAM) is needed for deployment (Section 3.4).
- **Optimism toward the unknown.** Unexplored ground is treated as free (Section 4.4); the robot may enter regions it has never perceived. This is a deliberate trade-off against freezing.
- **No reverse motion.** With $v \geq 0$, a robot cornered in a risk cul-de-sac can only rotate out, and the map does not update during fast rotation, a corner case to keep in mind.
- **Manual risk scores.** The r_c are set by hand; two physically different surfaces sharing a class label get the same score, which is the motivation for the self-supervised extension.
- **Semantic zones have no approach gradient.** Only rigid obstacles ($R \geq 0.9$) get a clearance halo, so the robot may graze the edge of a wet/mud patch (where the true cost is genuinely zero).

CHAPTER 8

Sustainable Development, Ethics and Occupational Health & Safety

Following the ENSIL-ENSCI reporting requirements, this chapter discusses the sustainable-development, ethical and health-and-safety dimensions of the internship.

8.1 Occupational Health and Safety

The internship took place in a robotics laboratory (DIMEAS, Politecnico di Torino) and on a GPU workstation. The identified risks and the mitigation measures were:

- **Electrical and battery risk.** The TurtleBot 4 (iRobot Create 3 base) uses a lithium-ion battery charged on its dock. Standard handling was applied: charging on the dock only, visual inspection for any swelling or damage, no short-circuit, and the workstation and chargers were used on protected mains outlets.
- **Moving robot.** The robot moves at $\leq 0.31\,\mathrm{m\,s^{-1}}$ (low kinetic energy). Physical tests were run in a delimited area with bystanders kept clear. The platform's cliff and bump sensors, together with the controller's software safety (a zero velocity command is published whenever an input is missing or the goal is reached), limit the consequences of a collision.
- **Workstation ergonomics and heat.** Prolonged screen work was managed with regular breaks and a suitable posture. The GPU (RTX 5070) heats up during training, so adequate ventilation was ensured.

- **Safety by simulation.** Crucially, most of the evaluation was carried out in *simulation*. This removes physical risk entirely: for instance, the robot's behaviour near *staircases* (a fall hazard) could be tested repeatedly without any risk to the hardware or to people. This is a direct safety benefit of the chosen methodology.

8.2 Sustainable Development

- **Energy footprint.** Deep-learning training is energy-intensive. The project deliberately deploys a *lightweight* network (YOLO26n, 1.6 M parameters) rather than a large one: it is several times faster at inference (about 46 vs 7 FPS on CPU), which drastically reduces energy use during operation and enables on-board, GPU-free deployment.
- **Resource use.** Evaluating in simulation rather than through repeated physical runs avoids battery cycles, mechanical wear and travel, and lets a large number of experiments be run at low material cost.
- **Hardware longevity.** The very purpose of risk-aware navigation, steering the robot away from wet floors, steps and stairs, reduces falls and damage, extending the hardware's life and limiting electronic waste.
- **Reuse.** The stack is built entirely on open-source tools (ROS 2, Gazebo, PyTorch, the MPPI-Generic library) and is documented, so it can be reused and extended without redevelopment.

8.3 Ethics and Scientific Integrity

- **Scientific integrity.** All external methods, libraries and works are cited (see the bibliography). Results are reported honestly, including what did *not* succeed (on-board perception on the real robot via the ghost-twin), and preliminary data are clearly distinguished from a controlled campaign, avoiding any misleading over-claim.
- **Data.** The training dataset is synthetic (simulation renders) and contains no personal data; the camera operates on indoor ground terrain, raising no privacy concern in this setting.
- **Precautionary approach.** Terrain that is genuinely non-traversable (staircases, walls) is treated as a hard barrier in the cost function, reflecting a precautionary stance appropriate to safety-relevant navigation.

CHAPTER 9

Conclusion and Future Work

9.1 Summary

This internship built and validated a complete *risk-aware* navigation stack for a TurtleBot 4, addressing a concrete perception gap: hazards that are geometrically flat but physically dangerous, invisible to a 2D LiDAR. The two contributions are a **two-layer terrain risk map**: a semantic layer ($R(\boldsymbol{p}) = \sum_c r_c \mathbb{P}(c \mid \boldsymbol{p})$)

fused with a geometric height layer and accumulated into a persistent BEV costmap and a **GPU risk-aware MPPI controller** that treats this costmap as a continuous cost field, tunable from risk-blind to strongly risk-averse through the single parameter λ_{risk}.

Beyond the implementation, the project produced a *quantitative* model of the controller's behaviour: an explicit cost-arbitrage calculation that predicts, for each hazard and geometry, the threshold at which the robot switches from crossing to detouring, and an ESS-based recipe for the softmax temperature. This turned tuning from guesswork into calculation and gave the results a falsifiable backbone.

Finally, the controller was demonstrated on the physical robot: driven by a ground-truth risk map baked into the costmap, the real TurtleBot 4 navigated a virtual hazard field, avoiding obstacles that existed only in the map: a controlled, repeatable, and hardware-safe way to validate risk-aware motion. Closing the loop with on-board camera perception *on the real robot* was attempted but not completed, and is the first item of future work.

9.2 Future Work

- **On-board perception on the real robot.** Complete the "ghost-twin" integration (Section 6.3.3), or, better, run the real camera through `risk_fusion` directly on the robot, so the physical TurtleBot 4 navigates from its *own* perception rather than a baked map. Both blocks (perception, control) already work independently; what remains is the spawn/DDS integration that blocked it.
- **Self-supervised risk learning.** Replace (or complement) the hand-set r_c with a STERLING-style [2] model that learns terrain risk from the robot's own IMU/odometry experience: all sensors are already available on the platform. This directly addresses the "manual scores" limitation.
- **Metric localisation.** Integrate AMCL or a SLAM stack so the real robot no longer depends on drifting wheel odometry, making the world-frame anchoring self-contained.
- **LiDAR into the geometric layer.** Fold the RPLIDAR returns into the geometric layer for full 360° obstacle coverage beyond the camera field of view.
- **Embedded deployment.** Port the controller to a smaller rollout count (≈ 1024) and the segmentation to the ONNX MobileNet model for on-board (Raspberry Pi / OAK-D VPU) execution, and measure the accuracy/latency trade-off.
- **Uncertainty-aware risk.** Use the segmentation's predictive entropy as an explicit uncertainty channel, giving the controller a principled cost for the unknown instead of the current optimism.
- **Full experimental campaign.** Complete the parameter sweeps of Chapter 6 on real hardware to quantify the safety-vs-efficiency trade-off across hazards.
- **Remaining secondary studies.** Run the pink-vs-white noise detour-discovery comparison and the cost-shaping ablation ($\alpha \in \{1.0, 0.9, 0.5\}$, Section 5.5), which were designed but not executed within the internship (Section 7.4).

9.3 Personal Takeaways

Beyond the technical outcome, this internship was a substantial learning experience. I gained hands-on proficiency with **ROS 2**, now an unavoidable framework in robotics, and deepened my day-to-day use of **Linux** as a development environment. On the perception side, I learned to train a semantic segmentation model end to end and, in doing so, to clearly distinguish *semantic* segmentation (labelling terrain *type*) from

instance segmentation (separating individual object instances), a distinction that directly shaped the design of the two-layer risk map. On the control side, I came to understand sampling-based predictive control in practice through MPPI: how a stochastic optimizer can be reasoned about and tuned quantitatively rather than by trial and error alone. Finally, working for several months in Italy, at DIMEAS, Politecnico di Torino, was also a personal immersion that improved both my English (the working language of the team) and my Italian. If I were to do it again, I would prioritise the perception-control integration on the real robot earlier in the schedule, since it turned out to be the most time-sensitive and failure-prone part of the project.

Bibliography

[1] Papadakis P. Terrain traversability analysis methods for unmanned ground vehicles: a survey. Eng Appl Artif Intell. 2013;26(4):1373–85.

[2] Karnan H, Yang E, Farkash D, Warnell G, Biswas J, Stone P. STERLING: self-supervised terrain representation learning from unconstrained robot experience. In: Conference on Robot Learning (CoRL); 2023. arXiv:2309.15302.

[3] Williams G, Aldrich A, Theodorou EA. Model predictive path integral control: from theory to parallel computation. J Guid Control Dyn. 2017;40(2):344–57.

[4] ACDSLab. MPPI-Generic: a CUDA library for model predictive path integral control [Internet]. 2024 [cited 2026 Aug 21]. Available from: `https://github.com/ACDSLab/MPPI-Generic`

APPENDIX A

Appendix

A.1 Default Controller Parameters

Table A.1 lists the default parameters of `mppi_node`, the retained tuning used throughout the Chapter 7 evaluation campaign. All are ROS parameters settable at launch, except T and K, which are compile-time template constants. Some evaluation runs override `w_goal` to 10.0 for the reasons discussed in Section 7.3.

Table A.1: Default MPPI controller parameters.

Parameter	Default	Meaning
`lambda_risk`	6.1	risk weight λ_{risk} (0 = baseline)
`lambda_temp`	0.85	softmax temperature (drive by ESS)
`w_goal`	10.0	goal-distance weight (per step)
`w_terminal`	10.0	terminal goal weight multiplier
`crash_threshold`	0.95	R above which a cell is a hard barrier
`v_max`	0.30	max linear speed ($\mathrm{m\,s^{-1}}$)
`w_max`	1.50	max angular speed ($\mathrm{rad\,s^{-1}}$)
`std_dev_v`	0.15	sampling σ_v
`std_dev_w`	0.50	sampling σ_ω
`alpha`	1.0	1 = pure cost-shaping
`noise_exponent_v/w`	1.0	0 = white, 1 = pink, 2 = brown
`control_hz`	14	loop rate; horizon (s) = T / `control_hz`
T (`TIMESTEPS`)	200	horizon length (compile-time)
K (`NUM_ROLLOUTS`)	5120	rollouts per cycle (compile-time)

A.2 Software and Reproduction

- **Prerequisites:** Ubuntu with ROS 2 Jazzy and Gazebo Harmonic installed and sourced, and a CUDA-capable GPU with the CUDA toolkit for the controller.
- **Clone:** the CUDA controller depends on the header-only MPPI-Generic library, versioned as a *git submodule* under `extern/MPPI-Generic`. Clone with `git clone -recurse-submodules`, or run `git submodule update -init` after a plain clone; a submodule left empty makes `risk_mppi_controller` fail to build.
- **Build:** `./build.sh` from the repository root (uses the system Python; the training `.venv` lacks `catkin_pkg` and breaks a clean colcon build).
- **Controller:** `risk_mppi_controller` (C++/CUDA), built on the header-only MPPI-Generic library under `extern/`.
- **Perception & sim:** `risk_mppi_sim` (Python, ROS 2 Jazzy, Gazebo Harmonic).
- **A baked scenario in simulation:** `ros2 launch risk_mppi_sim virtual_scenario_sim.launch.py layout:=cross_or_detour`, then set λ_{risk} and give a goal in RViz.
- **Real robot, virtual obstacles:** `ros2 launch risk_mppi_sim virtual_scenario_real.launch.py layout:=zigzag` (after sourcing the robot environment).
- **Source code:** the full implementation (perception, controller, simulation worlds, evaluation scripts) is version-controlled at `https://github.com/STREAMRobotics/risk_aware_mppi`, commit `3236907`, which reflects the parameters and scenarios actually used to produce the results of Chapter 7. The repository is currently private; access can be granted on request.

Résumé

Les robots mobiles d'intérieur doivent éviter des dangers qu'un LiDAR 2D ne peut pas percevoir : sol mouillé, boue, moquette épaisse, et d'autres surfaces géométriquement planes mais physiquement dangereuses. Ce stage développe une chaîne de navigation *consciente du risque* pour un TurtleBot 4. Une carte de risque à deux couches fuse une couche sémantique (un réseau de segmentation qui transforme chaque pixel caméra en un risque attendu $R(\boldsymbol{p}) = \sum_c r_c \, \mathbb{P}(c \mid \boldsymbol{p})$) avec une couche géométrique (obstacles issus du nuage de points de profondeur), accumulées dans une carte de coût persistante en vue de dessus. Un contrôleur GPU Model Predictive Path Integral (MPPI) traite cette carte comme un champ de coût continu et arbitre la progression vers l'objectif contre l'exposition au risque du terrain, réglée par un unique paramètre λ_{risk} (du risque-aveugle au fortement risque-averse). Un modèle explicite d'arbitrage de coût prédit le seuil de bascule traversée-contournement. Le système est validé en simulation (un monde entrepôt et des scénarios contrôlés) et démontré sur le robot physique naviguant un champ de risque virtuel. Sur deux scénarios contrôlés, régler λ_{risk} du réglage risque-aveugle au réglage risque-averse réduit l'exposition au risque réel de **44 %** sur une traversée forcée et de **100 %** sur une zone évitable en champ ouvert, pour un surcoût de distance et de temps de seulement 2–9 %.

Mots clés : navigation consciente du risque, franchissabilité, MPPI, segmentation sémantique, perception RGB-D, ROS 2, TurtleBot 4.

Abstract

Indoor mobile robots must avoid hazards a 2D LiDAR cannot perceive: wet tiles, mud, thick carpet, and other surfaces that are flat yet dangerous. This internship develops a *risk-aware* navigation stack for a TurtleBot 4. A two-layer risk map fuses a semantic layer (a segmentation network turning each pixel into an expected risk $R(\boldsymbol{p}) = \sum_c r_c \, \mathbb{P}(c \mid \boldsymbol{p})$) with a geometric layer (obstacles from the depth cloud), accumulated into a bird's-eye-view cost map. A GPU Model Predictive Path Integral (MPPI) controller treats this map as a continuous cost field and trades progress toward the goal against terrain-risk exposure, tuned by a single parameter λ_{risk} (from risk-blind to risk-averse). An explicit cost-arbitrage model predicts the crossing-versus-detour threshold. The system is validated in simulation (a warehouse world and controlled scenarios) and demonstrated on the physical robot navigating a virtual risk field. Across two controlled scenarios, tuning λ_{risk} from risk-blind to risk-aware cut ground-truth risk exposure by **44%** on a forced crossing and by **100%** on an open-field detourable patch, at a path-length and time cost of only 2–9%.

Keywords: risk-aware navigation, traversability, MPPI, semantic segmentation, RGB-D perception, ROS 2, TurtleBot 4.